

2025-J组 解题报告

难度系数：4.75

1 打字-字符串简单处理

思维难度：1

代码难度：2

解法分析

字符串处理，注意所有情况如：*A* 是一个名字，*abCde* 不是。

参考代码

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  string s;
4  int n;
5  int main()
6  {
7      //freopen("imena.in","r",stdin);
8      //freopen("imena.out","w",stdout);
9      cin>>n;
10     getchar();
11     getline(cin,s);
12     int len=s.size();
13     int ans=0;
14     int i=0;
15     while(i<len){
16         int flag=0;
17         if(s[i]>='A'&&s[i]<='Z'){
18             if(i>0&&s[i-1]!=' ') break;
19             int t=i+1;
20             while(s[t]!='.'&&s[t]!='.'&&s[t]!='!&&s[t]!='?'){
21                 if(s[t]>='a'&&s[t]<='z') t++;
22                 else {flag=1;break;}
23             }
24             if(flag==0) ans++;
25             i=t;
26         }
27         if(s[i]=='.'||s[i]=='?'||s[i]=='!'){
28             cout<<ans<<endl;
29             ans=0;
30         }
31         i++;
32     }
33     return 0;
34 }
```

2 下棋-枚举

思维难度：2

代码难度：2

解法分析

50分

贪心，因为数据规定每个格子右边和下边的两个英文字母不同，依次选择字典序最小的字母即可。

70分

搜索

100分

枚举

对于每一步，步数相同时，坐标 (X, Y) ， $X + Y$ 的值相同。所以基本思路是：枚举每一步中的所有字母，同时需要判断上一步是否可到达(才能走到当前位置)，选出其中字典序最小的，标记这些位置是可达的，这些位置可以进行扩展。

参考代码

```
1  #include<bits/stdc++.h>
2  #define N 2100
3  using namespace std;
4  char a[N][N];
5  int n,m,vis[N][N]; //vis[i][j]标记a[i][j]是否可以继续往后扩展
6  int main()
7  {
8      //freopen("pohlepko.in","r",stdin);
9      //freopen("pohlepko.out","w",stdout);
10     scanf("%d%d",&n,&m);
11     for(int i=0;i<=n;i++)
12         for(int j=0;j<=m;j++)
13             a[i][j]='z'; //初始化为z
14     for(int i=1;i<=n;i++) scanf("%s",a[i]+1);
15     cout<<a[1][1];
16     vis[1][1]=1;
17     for(int k=2;k<=n+m-1;k++){ // 枚举第k个字母
18         char ch='z';
19         for(int x=1;x<=n;x++){ //枚举第k个字母的横坐标
20             int y=k+1-x; //算出纵坐标
21             if(y>=1&&y<=m){
22                 if(vis[x-1][y]&&a[x][y]<ch) ch=a[x][y];
23                 if(vis[x][y-1]&&a[x][y]<ch) ch=a[x][y];
24             }
25         }
26         cout<<ch; //输出最小字母
27         for(int x=1;x<=n;x++){ //枚举第k个字母的横坐标
28             int y=k+1-x; //算出纵坐标
29             if(y>=1&&y<=m) { //如果等于最小字母，就可以继续向下扩展
30                 if(vis[x-1][y]&&a[x][y]==ch) vis[x][y]=1;
31                 if(vis[x][y-1]&&a[x][y]==ch) vis[x][y]=1;
32             }
33         }
34     }
```

```
35     return 0;  
36 }
```

3 纸牌大归位-模拟+优化

思维难度：3

代码难度：3

解法分析

- 10 – 20分

模拟，列举所有情况，枚举端点，进行翻转。时间复杂度 $O(N^3)$

```
1 For(左端点)
2     For(右端点)
3         翻转
```

- 100分

- 0、先设定一些概念：**归位**，就是数字 t 回到 t 号位置，归位后的点，叫 **固定点**；
- 1、思考可得：每个号码，都只能通过一个 **轴心** 进行 **归位**。
- 2、先计算出每一个号码 **归位** 需要的 **轴心**，以及距离 **轴心** x 的位置。
- 3、处理离 **轴心** x 最近的号码，这样，可以直接使用桶计数，让其围绕轴心旋转后得到的固定点数量 $sum[x]++$ ，同时记录左右端点。
- 4、在这一个区间通过旋转得到的**固定点数目=旋转后得到的固定点数目-未旋转时的固定点数目**。其中未旋转的固定点数目可以使用**前缀和**的方法进行统计，在 $O(1)$ 的时间求得。
- 5、对每一个号码，进行旋转时，记录下其左右端点，以及得到的固定点数目，最后进行排序即可。
- 6、对于轴心为小数的可以单独用数组处理，也可以将数组扩大，如 1.5 对应 $N+1$ ，2.5 对应 $N+2$
- 7、处理每一个号码的时间复杂度为 $O(N)$ ，排序的时间复杂度为 $O(N \log N)$ ，所以总的时间复杂度为 $O(N \log N)$ 。

参考代码

```
1  #include<bits/stdc++.h>
2  #define N 500100
3  using namespace std;
4  int n,a[N];
5  int sum[2*N],s[N]; //s为前缀和
6  struct node{
7      int x,y,z;
8  }ans[N];
9  bool comp2(node p,node q){
10     if(p.z<q.z) return 0;
11     if(p.z>q.z) return 1;
12     if(p.y-p.x<q.y-q.x) return 1;
13     if(p.y-p.x>q.y-q.x) return 0;
14     if(p.x<q.x) return 1;
15     return 0;
16 }
17 struct node2{
18     int k,d,ss;
19 }num[N];
20 bool comp(node2 p,node2 q){
21     if(p.d<q.d) return 1;
22     return 0;
```

```

23 }
24 int main(){
25     // freopen("doktor.in","r",stdin);
26     // freopen("doktor.out","w",stdout);
27     scanf("%d",&n);
28     for(int i=1;i<=n;i++){
29         scanf("%d",&a[i]);
30         if(a[i]==i) s[i]=s[i-1]+1; //未交换之前的固定点数量前缀和
31         else s[i]=s[i-1];
32         num[i].k=i; //归位位置
33         num[i].d=abs(i-a[i])/2; //与轴的距离
34         num[i].ss=a[i]; //初始位置
35     }
36     sort(num+1,num+n+1,comp); //以d作为关键字排序
37     //处理轴心x
38     for(int i=1;i<=n;i++){
39         int mx=max(num[i].k,num[i].ss);
40         int mn=min(num[i].k,num[i].ss); //左右端点
41         if((mx-mn)%2==0){ //以mn+(mx-mn)/2为轴，x能够变成固定数目
42             int tmp=mn+(mx-mn)/2;
43             sum[tmp]++;
44             ans[i].x=mn;
45             ans[i].y=mx;
46             ans[i].z=sum[tmp]-(s[mx]-s[mn-1]);
47         }
48         else {
49             int tmp=mn+n+(mx-mn)/2; //对于1.5,记为N+1, 2.5记为N+2..以此类推
50             sum[tmp]++;
51             ans[i].x=mn;
52             ans[i].y=mx;
53             ans[i].z=sum[tmp]-(s[mx]-s[mn-1]);
54         }
55     }
56     sort(ans+1,ans+n+1,comp2);
57     printf("%d %d\n",a[ans[1].x],a[ans[1].y]);
58     return 0;
59 }

```

4 分钱-DP+优化

思维难度：4

代码难度：2

解法分析

- 50分

爆搜，列举出所有分配方案，时间复杂度为 $O(2^{13})$ ；

- 70分

DP

$dp[k][i][j]$ ，表示前 k 张钞票，第一个人分得金额 i ，第二个人分得金额 j 。

1 表示成立，0 表示不成立，也是类似的将所有情况都列举出来。

第 k 张钞票金额 $a[k]$ 有三种情况：

①第 k 张分给第一个人

```
1 | if(a[k]<=i && dp[k-1][i-a[k]][j]==1) dp[k][i][j]=1;
```

②第 k 张分给第二个人

```
1 | if(a[k]<=j && dp[k-1][i][j-a[k]]==1) dp[k][i][j]=1;
```

③都不给

```
1 | if(dp[k-1][i][j]==1) dp[k][i][j]=1;
```

最后找到 $dp[n][i][i]$ 中最大的 i 值即可。

- 100分

DP

对70分的DP进行优化。

$dp[i][j]$ 表示前 i 张钞票分后两人的差值为 j 的最大总金额

第 k 张钞票金额 $a[k]$ 有四种情况：

①都不给

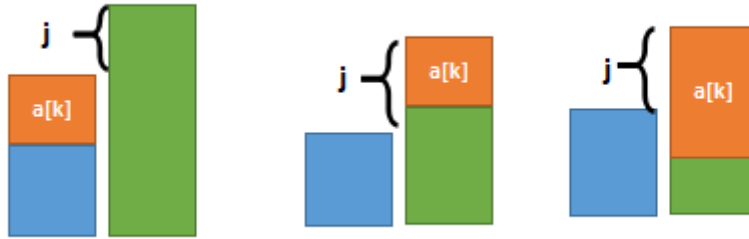
$$dp[i][j] = dp[i-1][j];$$

②下图三种情况

$$dp[i][j] = \max(dp[i][j], dp[i-1][j+a[k]] + a[k]);$$

$$dp[i][j] = \max(dp[i][j], dp[i-1][j-a[k]] + a[k]);$$

$$dp[i][j] = \max(dp[i][j], dp[i-1][a[k]-j] + a[k]);$$



参考代码

```

1  #include<bits/stdc++.h>
2  #define N 510
3  using namespace std;
4  int dp[N][101000]; //dp[i][j]表示前i张钞票分后两人的差值j的最大总金额
5  int n;
6  int a[N];
7  int main(){
8      //freopen("kas.in","r",stdin);
9      //freopen("kas.out","w",stdout);
10     scanf("%d",&n);
11     int sum=0;
12     for(int i=1;i<=n;i++){
13         scanf("%d",&a[i]);
14         sum+=a[i];
15     }
16     memset(dp,128,sizeof(dp));
17     dp[0][0]=0;
18     for(int i=1;i<=n;i++){
19         for(int j=0;j<=sum;j++){
20             //都不给
21             dp[i][j]=dp[i-1][j];
22             //第k张分给金额少的一个人
23             dp[i][j]=max(dp[i][j],dp[i-1][j+a[i]]+a[i]);
24             //第k张分给金额多的一个人
25             dp[i][j]=max(dp[i][j],dp[i-1][abs(j-a[i])] +a[i]);
26         }
27     }
28     cout<<sum-dp[n][0]/2;
29     return 0;
30 }

```